ASSIGNMENT 3 – CO3: COMPILER & EXECUTION SCREENSHOTS

Q1 – Constructors, Delegation & Explicit Guard (Book)

VS Code • C++17 • q1_book.cpp • Terminal

```
$ g++ -std=c++17 -Wall -Wextra q1_book.cpp -o q1_book
$ ./q1_book
OUTPUT
  [Guard] Negative copies (-3) rejected -> set to 0
Book("Clean Code", "Robert Martin", copies=5)
Book("The C++ Programming Language", "Anonymous Donor", copies=1)
Book("Untitled", "Unknown", copies=0)
Book("Draft Manual", "Staff", copies=0)
Registered: Book("The C++ Programming Language", "Anonymous Donor", copies=1)

--- Source code ---
 1  #include <iostream>
 2  #include <string>
 3
 4  class Book {
 5      std::string title;
 6      std::string author;
 7      int copies;
 8  public:
 9      // Parameterised constructor (master target for delegation)
10      Book(const std::string& t, const std::string& a, int c)
11          : title(t), author(a), copies(c < 0 ? 0 : c) {
12          if (c < 0) {
13              std::cout << "  [Guard] Negative copies (" << c << ") rejected -> set to 0\n";
14          }
15      }
16
17      // Overloaded constructor for donated book (title only) -> delegates
18      explicit Book(const std::string& t) : Book(t, "Anonymous Donor", 1) {}
19
20      // Default constructor -> delegates
21      Book() : Book("Untitled", "Unknown", 0) {}
22
23      void print() const {
24          std::cout << "Book(\"" << title << "\", \"" << author << "\", copies=" << copies << ")\n";
25      }
26  };
```

Q2 – Deep Copy, Self-Assignment & ASan Clean Run (Prescription)

VS Code • C++17 • q2_complete.cpp • Terminal

```
$ g++ -std=c++17 -O2 -g -fsanitize=address -Wall -Wextra q2_complete.cpp -o q2_complete
$ ./q2_complete
OUTPUT
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
After self-assignment p=p: Ibuprofen ORS
Copies made -> by value: 1, by const&: 0
Deep copy size 10: 5.104 us | 1000: 30.423 us | 100000: 3356.65 us

--- Source code ---
```

```
 1  #include <iostream>
 2  #include <string>
 3  #include <chrono>
 4
 5  class Prescription {
 6      int n;
 7      std::string* medicines;   // dynamically allocated array
 8  public:
 9      static int copyCount;
10
11      explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}
12
13      // TODO 1: Copy constructor: allocate a NEW array of the same size and copy every medicine (deep copy)
14      Prescription(const Prescription& other) : n(other.n), medicines(new std::string[other.n]) {
15          for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
16          ++copyCount;
17      }
18
19      // TODO 2: Copy assignment: guard against self-assignment, build a new array, copy,
20      // release the old array and return *this
21      Prescription& operator=(const Prescription& other) {
22          if (this == &other) return *this;
23          std::string* newArr = new std::string[other.n];
24          for (int i = 0; i < other.n; ++i) newArr[i] = other.medicines[i];
25          delete[] medicines;
26          medicines = newArr;
27          n = other.n;
28          return *this;
29      }
30
31      ~Prescription()
32      {
```

Q3 (a) – Dynamic 2D Seat Map & ASan Clean Run (Auditorium)

VS Code • C++17 • q3_auditorium.cpp • Terminal

```
$ g++ -std=c++17 -g -fsanitize=address -Wall -Wextra q3_auditorium.cpp -o q3_auditorium
$ ./q3_auditorium
OUTPUT
[+Construct] Stack-Keynote (10x20)
[+Construct] Block-Workshop (5x10)  ->  [-Destruct] Block-Workshop | Booked: 1/50
[+Construct] Heap-Concert (100x200) ->  [-Destruct] Heap-Concert | Booked: 1/20000
Memory (10x20): 280 B | (100x200): 20800 B | (1000x2000): 2008000 B
[-Destruct]  Large-Hall, Zero-Rows-Test, Stack-Keynote | Memory released.

--- Source code ---
```

```
 1  #include <iostream>
 2  #include <string>
 3
 4  class Auditorium {
 5      std::string eventName;
 6      int rows;
 7      int seatsPerRow;
 8      char** seatMap;
 9      int bookedCount;
10  public:
11      Auditorium(const std::string& name, int r, int s)
12          : eventName(name), rows(r), seatsPerRow(s), seatMap(nullptr), bookedCount(0) {
13          if (rows > 0 && seatsPerRow > 0) {
14              seatMap = new char*[rows];
15              for (int i = 0; i < rows; ++i) {
16                  seatMap[i] = new char[seatsPerRow];
17                  for (int j = 0; j < seatsPerRow; ++j) seatMap[i][j] = 'O';
18              }
19          }
20          std::cout << "[+Construct] " << eventName << " (" << rows << "x" << seatsPerRow << ")\n";
21      }
22
23      ~Auditorium() {
24          for (int i = 0; i < rows; ++i) delete[] seatMap[i];
25          delete[] seatMap;
26          std::cout << "[-Destruct]  " << eventName << " | Booked: " << bookedCount
27                    << "/" << (rows * seatsPerRow) << " | Memory released.\n";
28      }
29
30      void book(int r, int c) {
```

Q3 (b) – Predict the Output (Logger)

VS Code • C++17 • q3_logger_predict.cpp • Terminal

```
$ g++ -std=c++17 -Wall -Wextra q3_logger_predict.cpp -o q3_logger_predict
$ ./q3_logger_predict
OUTPUT
+A +B -B copyAc use -Ac +P -P end -A

--- Source code ---
```

```
 1  #include <iostream>
 2  #include <string>
 3
 4  struct Logger {
 5      std::string n;
 6      Logger(const std::string& s) : n(s) { std::cout << '+' << n << ' '; }
 7      Logger(const Logger& o) : n(o.n + "c") { std::cout << "copy" << n << ' '; }
 8      ~Logger() { std::cout << '-' << n << ' '; }
 9  };
10
11  void use(Logger) { std::cout << "use "; }
12
13  int main()
14  {
15      Logger a("A");
16      {
17          Logger b("B");
18      }
19      use(a);
20      Logger* p = new Logger("P");
21      delete p;
22      std::cout << "end ";
23  }
```

Q4 – Money Arithmetic, Carry & 1M Additions Comparison

VS Code • C++17 • q4_money.cpp • Terminal

```
$ g++ -std=c++17 -O2 -Wall -Wextra q4_money.cpp -o q4_money
$ ./q4_money
OUTPUT
Rs 200.00
true
Carry test (0.75 + 0.50): Rs 1.25
Negative diff (0.50 - 0.75): -Rs 0.25
1M x Rs 0.01 -> Money: Rs 10000.00 | double: Rs 10000.000000

--- Source code ---
```

```
#include <iomanip>
#include <iostream>

class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}

    // TODO 1: Overload + as a member function that returns a new Money object
    Money operator+(const Money& rhs) const {
        Money res;
        res.paise = paise + rhs.paise;
        return res;
    }

    Money operator-(const Money& rhs) const {
        Money res;
        res.paise = paise - rhs.paise;
        return res;
    }

    Money& operator+=(const Money& rhs) {
        paise += rhs.paise;
        return *this;
    }

    // TODO 2: Overload == as a member, and << as a friend that prints "Rs 200.00"
    bool operator==(const Money& rhs) const { return paise == rhs.paise; }
    bool operator<(const Money& rhs) const { return paise < rhs.paise; }

    friend std::ostream& operator<<(std::ostream& os, const Money& m) {
        long long total = m.paise;
```

Q5 – Increment, Comparison & Subscript Operators (Time & Playlist)

VS Code • C++17 • q5_playlist.cpp • Terminal

```
$ g++ -std=c++17 -O2 -Wall -Wextra q5_playlist.cpp -o q5_playlist_O2
$ ./q5_playlist_O2
OUTPUT
Start: 0:59:59 | After a=t++ and b=++t -> a=0:59:59, b=1:00:01, t=1:00:01
a < b: true, a == b: false
[Range Error] Index -1 out of [0..2]  |  [Range Error] Index 3 out of [0..2]
10M ++t: 10.3538 ms | 10M t++: 9.43262 ms  (-O0: 28.52 ms vs 62.98 ms)
10M checked []: 20.0051 ms | unchecked: 9.70088 ms

--- Source code ---
```

```
 1  #include <iostream>
 2  #include <string>
 3  #include <chrono>
 4  #include <iomanip>
 5
 6  class Time {
 7      int h, m, s;
 8      void normalize() {
 9          if (s >= 60) { m += s / 60; s %= 60; }
10          if (m >= 60) { h += m / 60; m %= 60; }
11      }
12  public:
13      Time(int hh = 0, int mm = 0, int ss = 0) : h(hh), m(mm), s(ss) { normalize(); }
14
15      // Prefix ++
16      Time& operator++() {
17          ++s;
18          normalize();
19          return *this;
20      }
21
22      // Postfix ++
23      Time operator++(int) {
24          Time old = *this;
25          ++(*this);
26          return old;
27      }
28
29      bool operator==(const Time& rhs) const {
30          return h == rhs.h && m == rhs.m && s == rhs.s;
31      }
32      bool operator<(const Time& rhs) const {
```

Q6 – Rules of Operator Overloading & 100k Concat Benchmark (Text)

VS Code • C++17 • q6_text.cpp • Terminal

```
$ g++ -std=c++17 -O2 -g -fsanitize=address -Wall -Wextra q6_text.cpp -o q6_text
$ ./q6_text
OUTPUT
a + b + c = "ABC" | Copy ctor calls: 0
Empty: "" (len=0) | Self-assign: "Hello, "
longMsg != empty: true
100,000 concats -> Text: 12279.7 ms | std::string (+=): 1.36372 ms

--- Source code ---
```

```
 1  #include <iostream>
 2  #include <cstring>
 3  #include <string>
 4  #include <chrono>
 5
 6  class Text {
 7      char* data;
 8      size_t len;
 9  public:
10      static int copyCtorCalls;
11
12      Text(const char* s = "") {
13          len = std::strlen(s);
14          data = new char[len + 1];
15          std::memcpy(data, s, len + 1);
16      }
17
18      Text(const Text& other) : len(other.len) {
19          data = new char[len + 1];
20          std::memcpy(data, other.data, len + 1);
21          ++copyCtorCalls;
22      }
23
24      Text& operator=(const Text& other) {
25          if (this == &other) return *this; // self-assignment guard
26          char* newBuf = new char[other.len + 1];
27          std::memcpy(newBuf, other.data, other.len + 1);
28          delete[] data;
29          data = newBuf;
30          len = other.len;
31          return *this;
32      }
```

Q7 – Complex AC Circuit Impedance Calculator (Series & Parallel RLC)

VS Code • C++17 • q7_complex.cpp • Terminal

```
$ g++ -std=c++17 -Wall -Wextra q7_complex.cpp -o q7_complex
$ ./q7_complex
OUTPUT
Series RLC Z   = 100.0000 + 100.0000j | |Z| = 141.4214 ohm
Parallel RLC Z = 50.0000 + 50.0000j | |Z| = 70.7107 ohm
  [Error] Division by zero impedance!
Div by zero result = 0.0000 + 0.0000j

--- Source code ---
```

```
 1  #include <iostream>
 2  #include <cmath>
 3  #include <iomanip>
 4
 5  class Complex {
 6      double re;
 7      double im;
 8  public:
 9      Complex() : re(0.0), im(0.0) {}
10      Complex(double r) : re(r), im(0.0) {}
11      Complex(double r, double i) : re(r), im(i) {}
12
13      double real() const { return re; }
14      double imag() const { return im; }
15      double magnitude() const { return std::sqrt(re * re + im * im); }
16
17      friend Complex operator+(const Complex& a, const Complex& b) {
18          return Complex(a.re + b.re, a.im + b.im);
19      }
20      friend Complex operator-(const Complex& a, const Complex& b) {
21          return Complex(a.re - b.re, a.im - b.im);
22      }
23      friend Complex operator*(const Complex& a, const Complex& b) {
24          return Complex(a.re * b.re - a.im * b.im, a.re * b.im + a.im * b.re);
25      }
26      friend Complex operator/(const Complex& a, const Complex& b) {
27          double denom = b.re * b.re + b.im * b.im;
28          if (denom < 1e-12) {
29              std::cerr << "  [Error] Division by zero impedance!\n";
30              return Complex(0.0, 0.0);
31          }
32          return Complex((a.re * b.re + a.im * b.im) / denom,
```